AI & Machine Learning Engineering

Master Interview Questions & Comprehensive Reference Guide

This master document synthesizes 52 curated interview questions and detailed solutions spanning 7 primary domains: **Machine Learning Fundamentals**, **Deep Learning & Neural Architectures**, **NLP & Large Language Models (LLMs)**, **Retrieval-Augmented Generation (RAG) & Vector Stores**, **Agentic AI Systems**, **MLOps, Dataiku & System Design**, and **Practical Coding & Troubleshooting**.

Key Highlights & Visual Enhancements:
• **Fresh Violet-Indigo Palette**: Completely redesigned with a vibrant Deep Violet and Charcoal Charcoal theme.
• **Enlarged Question Headers**: Prominent 12pt Bold Deep Violet titles for quick reference during revision.
• **High-Contrast Answer Formatting**: Styled using Graphite text to distinguish detailed explanations from question titles.
• **Clear Horizontal Dividers**: Border lines separating every question block across all pages.
• **Newly Added Topics**: Features enterprise platforms (Dataiku DSS explained simply), LLM VRAM math, non-LLM RAG metrics, and MCP protocols.

Domain Category	Question Count
1. Machine Learning Fundamentals	Q1 – Q10 (10 Questions)
2. Deep Learning & Neural Networks	Q11 – Q18 (8 Questions)
3. NLP, LLMs & Fine-Tuning	Q19 – Q26 (8 Questions)
4. RAG & Vector Databases	Q27 – Q35 (9 Questions)
5. Agentic AI Systems & Architecture	Q36 – Q40 (5 Questions)
6. MLOps, Dataiku & System Design	Q41 – Q46 (6 Questions)
7. Practical Coding, Debugging & Case Studies	Q47 – Q52 (6 Questions)
Total Comprehensive Questions	**52 Questions**

Version: 4.0 (Deep Violet Palette & Dataiku Integration) | **Format:** Master Technical Cheatsheet

1. Machine Learning Fundamentals

Q1. Explain the bias-variance tradeoff. How do you diagnose which one is affecting your model?

• **Bias**: Error introduced by approximating a complex real-world problem with an overly simplistic model (underfitting). Results in high error on both training and test datasets.
• **Variance**: Error introduced by the model's excessive sensitivity to small fluctuations in the training dataset (overfitting). Results in very low training error but high test error.
• **Diagnosis**: Plot learning curves (Training Error vs. Validation Error against dataset size or model complexity). High Bias shows high training error that levels off close to high validation error. High Variance shows a large gap between low training error and high validation error.

Q2. What is the difference between bagging and boosting? Give examples.

• **Bagging (Bootstrap Aggregating)**: Builds multiple independent estimators in parallel on random bootstrapped subsets of training data and averages predictions (or votes). Reduces variance. Example: *Random Forest*.
• **Boosting**: Builds weak learners sequentially, where each new model focuses on correcting the residual errors of prior models. Reduces bias primarily. Examples: *XGBoost, LightGBM, CatBoost*.
• **Key Tradeoff**: Bagging is hard to overfit and easy to parallelize; Boosting achieves higher accuracy but is sensitive to noise and hyperparameter choice.

Q3. How do you handle imbalanced datasets in production machine learning?

• **Resampling Techniques**: Synthetic minority oversampling (SMOTE/ADASYN) or strategic undersampling of the majority class.
• **Cost-Sensitive Learning**: Adjust class weights in the loss function (e.g., scale_pos_weight in XGBoost, class_weight='balanced' in scikit-learn).
• **Metric Selection**: Avoid Accuracy. Use Precision-Recall AUC (PR-AUC), F1-Score, or ROC-AUC.
• **Framing**: Reframe extreme imbalance (<1% positive class) as Anomaly Detection (e.g., Isolation Forests, One-Class SVM, or Autoencoders).

Q4. Explain precision, recall, and F1-score. When would you prioritize one over the other?

• **Precision** = TP / (TP + FP): Of all positive predictions, how many were actually positive? Prioritize when False Positives are expensive (e.g., spam detection blocking legitimate emails).
• **Recall** = TP / (TP + FN): Of all actual positive cases, how many were correctly caught? Prioritize when False Negatives are critical (e.g., cancer diagnosis, fraud detection).
• **F1-Score** = 2 * (Precision * Recall) / (Precision + Recall): Harmonic mean, providing a balanced single metric when both FP and FN matter.

Q5. What is regularization, and how do L1 (Lasso) and L2 (Ridge) differ?

• **Regularization** penalizes large weights in the loss function to prevent overfitting.
• **L1 (Lasso)** adds lambda * sum(|w_i|). Shrinks coefficients to exactly zero, producing sparse models and acting as an automatic feature selection mechanism.
• **L2 (Ridge)** adds lambda * sum(w_i^2). Shrinks coefficients smoothly toward zero but never eliminates them completely; handles multicollinearity effectively.
• **ElasticNet** combines both penalties (alpha * L1 + (1 - alpha) * L2).

Q6. How does a Random Forest differ from Gradient Boosted Trees?

• **Tree Depth**: Random Forest uses deep, unpruned trees (low bias, high variance); GBDT uses shallow trees / decision stumps (high bias, low variance).
• **Execution**: RF builds trees independently in parallel; GBDT builds sequentially based on gradients of the loss function.
• **Robustness**: RF works well out-of-the-box with default hyperparameters; GBDT requires tuning learning rate, subsample ratio, and tree depth.

Q7. What is cross-validation, and why is it important?

• Cross-validation splits data into k non-overlapping folds. The model is trained on k - 1 folds and validated on the remaining fold, repeating k times.
• Prevents optimistic bias from a single train-test split, provides a confidence interval for model performance, and ensures hyperparameter tuning generalizes well across different data distributions.

Q8. Explain the curse of dimensionality and how to address it.

• As feature count grows, data volume required to maintain data density grows exponentially. Distance metrics (Euclidean distance) become equidistant, making nearest-neighbor algorithms ineffective.
• **Solutions**: Dimensionality reduction (PCA, t-SNE, UMAP), feature selection (L1 regularization, recursive feature elimination), and tree-based models invariant to uninformative features.

Q9. What is the difference between generative and discriminative models?

• **Discriminative Models**: Learn the decision boundary P(y|x) directly. Examples: Logistic Regression, SVM, Gradient Boosting, Neural Network Classifiers.
• **Generative Models**: Learn joint probability P(x, y) = P(x|y)P(y) to model how data is generated. Can generate new synthetic samples. Examples: Naive Bayes, GANs, Variational Autoencoders (VAEs), Diffusion Models.

Q10. How do you choose the right evaluation metric for a business problem?

• Translate business costs into technical metrics. Determine the financial or operational cost of False Positives vs. False Negatives.
• If FP cost is high (e.g., declining good credit applications), optimize for High Precision.
• If FN cost is high (e.g., missing critical equipment failure), optimize for High Recall.
• Align technical metrics with business KPIs (e.g., Conversion Rate Lift, Customer LTV, Revenue per User) rather than default accuracy.

2. Deep Learning & Neural Networks

Q11. Explain backpropagation in simple terms.

• Backpropagation is the algorithm used to train neural networks. It calculates the partial derivative (gradient) of the loss function with respect to each weight using the mathematical **Chain Rule**.
• Gradients flow backward from the output layer to the input layer. An optimization algorithm (SGD, Adam) uses these gradients to update weights in the direction that minimizes loss (w_new = w_old - eta * grad_L).

Q12. What is the vanishing/exploding gradient problem, and how do you mitigate it?

• **Vanishing Gradients**: In deep networks, gradients become exponentially small as they propagate backward through many layers, causing early layers to stop learning (common with Sigmoid/Tanh activations).
• **Exploding Gradients**: Gradients grow exponentially large, causing unstable updates and numerical overflow NaN errors.
• **Mitigations**: ReLU/LeakyReLU activations, proper weight initialization (He/Xavier), Batch Normalization, Residual Skip Connections (ResNet), and Gradient Clipping.

Q13. Compare CNNs, RNNs, and Transformers — when would you use each?

• **CNNs (Convolutional Neural Networks)**: Capture spatial hierarchies using local convolutional filters. Best for images, spatial grids, and audio spectrograms.
• **RNNs/LSTMs**: Process sequential data step-by-step using a persistent hidden state. Best for short time-series data with strict sequential ordering.
• **Transformers**: Use self-attention mechanisms to model relationships across an entire sequence in parallel. Best for NLP, LLMs, vision transformers, and long-range sequential data.

Q14. What is the role of batch normalization?

• Batch Normalization normalizes the inputs of each layer within a mini-batch to have zero mean and unit variance (mean = 0, std = 1).
• Reduces internal covariate shift, speeds up convergence, allows higher learning rates, and acts as a mild regularizer.

Q15. Explain dropout and why it helps prevent overfitting.

• Dropout randomly zero-out a fraction p of neuron activations during each forward training pass.
• Forces the network to learn redundant representations without co-adapting on specific weights. Acts like training an ensemble of 2^N sub-networks. During inference, all neurons are active but scaled by (1 - p).

Q16. What is self-attention, and how does multi-head attention work (with CNN analogy)?

• **Self-Attention** computes similarity scores between all tokens in a sequence using Query (Q), Key (K), and Value (V) matrices: Softmax(QK^T / sqrt(d_k))V.
• **Multi-Head Attention** runs multiple attention projections in parallel, allowing the model to focus on different aspects of relationships (syntax, positional, semantic context) simultaneously.
• **CNN Analogy**: Multiple attention heads act like multiple convolutional kernels in CNNs—each kernel detects a different spatial feature (edges, textures), while each attention head learns a different semantic connection.

Q17. How does a Transformer differ from an LSTM architecturally?

• **LSTM**: Sequential recurrence (O(N) execution steps). Hard to parallelize on GPUs. Suffers from forgetting over very long context sequences.
• **Transformer**: Non-sequential self-attention (O(1) sequential execution steps). Fully parallelizable across tokens during training, enabling massive compute scaling.

Q18. What is transfer learning, and when would you use it vs. training from scratch?

• Transfer learning takes a model pre-trained on a massive dataset (e.g., ImageNet, Llama 3) and fine-tunes it on a target domain dataset.
• Use Transfer Learning when target dataset is small or compute budget is limited. Train from scratch only when domain is drastically different (e.g., satellite synthetic aperture radar) and labeled data is abundant.

3. NLP, LLMs & Fine-Tuning

Q19. How do you calculate GPU VRAM required for LLM inference?

Step-by-Step VRAM Calculation Formula for LLM Inference:
Total VRAM required is the sum of 3 main components:
1. **Model Weights Memory** = Parameters * Bytes_per_Parameter
2. **KV Cache Memory** = 2 * Layers * Hidden_Dim * Bytes_per_Parameter * Context_Length * Concurrent_Users
3. **Activation Buffers & Overhead** = ~10% of (Weights + KV Cache)

Worked Example (70 Billion Parameter Model, 16-bit FP16 precision, 10 concurrent users, 32K context):

```
1. Model Weight Memory:
70 Billion params * 2 bytes (FP16) = 140 GB

2. Key-Value (KV) Cache Memory:
Params per token per layer = 80 layers * 8192 hidden_dim * 2 (Key + Value) * 2 bytes = 2,621,440
bytes (~2.62 MB per token)
For 32,768 tokens * 10 concurrent users = 2.62 MB * 32,768 * 10 = 858.9 GB

3. Overhead & Temporary Buffers (10%):
(140 GB + 858.9 GB) * 10% = 99.8 GB

TOTAL INFERENCE VRAM REQUIRED = 140 + 858.9 + 99.8 = 1,098.7 GB (~1.1 TB VRAM)
```

Q20. When would you use LoRA fine-tuning vs. full fine-tuning?

• **LoRA (Low-Rank Adaptation)** freezes pre-trained model weights and injects trainable rank-decomposition matrices (A and B) into Transformer attention layers (W + Delta_W = W + B * A). Reduces trainable parameters by >99% and VRAM requirements by 60–70%.
• **Use LoRA** when adapting to a specific downstream task/style, GPU VRAM is limited, or hosting multiple specialized adapter weights on a single shared base model.
• **Use Full Fine-Tuning** when injecting vast new domain knowledge (e.g., medical/legal language pre-training) where low-rank approximations are insufficient.

Q21. How do you prevent catastrophic forgetting when fine-tuning an LLM?

• **Catastrophic Forgetting** occurs when a fine-tuned model forgets general pre-training knowledge while over-fitting to the fine-tuning dataset.
• **Mitigations**: Use LoRA (freezes base weights), incorporate a small percentage (5–10%) of general pre-training dataset mix into fine-tuning data, use low learning rates with warmup schedules, or apply Elastic Weight Consolidation (EWC).

Q22. How do you evaluate a model after fine-tuning?

• **Quantitative Automated Benchmarks**: Run standard benchmark suites (MMLU, GSM8K, HumanEval) to check general reasoning retention.
• **Domain-Specific Hold-Out Evaluation**: Compute Exact Match, F1-Score, or Task Accuracy on held-out validation data.
• **LLM-as-a-Judge**: Use stronger LLMs (e.g., GPT-4o) with structured rubric prompts to grade response quality, helpfulness, and style.
• **Human Side-by-Side (Elo Rating)**: Perform blind A/B preference testing between base model and fine-tuned model.

Q23. What are the key tradeoffs between Fine-Tuning, RAG, and Prompt Engineering?

• **Prompt Engineering**: No code/weights change, fast, zero training cost. Limited by context window and model capacity.
• **RAG (Retrieval-Augmented Generation)**: Connects LLM to external real-time data. Zero weight training needed, transparent source attribution, easy to update/delete documents.
• **Fine-Tuning**: Teaches the model new structure, tone, style, or syntax. High training cost, cannot easily update facts without retraining, prone to hallucinations if data is static.

Q24. Explain tokenization, context window, model size, and model capacity.

• **Tokenization**: Process of converting raw text into sub-word numerical IDs (e.g., Byte-Pair Encoding).
• **Context Window**: Maximum number of tokens an LLM can process in a single forward pass.
• **Model Size**: Number of trainable parameters (e.g., 7B, 70B). Determines memory footprint.
• **Model Capacity**: The expressiveness and knowledge storage limit of the architecture.

Q25. What is RLHF (Reinforcement Learning from Human Feedback) and why is it used?

• RLHF aligns pre-trained base LLMs with human expectations (making them Helpful, Honest, and Harmless).
• **Pipeline**: (1) Supervised Fine-Tuning (SFT) on prompt-response pairs --> (2) Train a Reward Model on human preference rankings --> (3) Optimize LLM policy via Proximal Policy Optimization (PPO) or Direct Preference Optimization (DPO).

Q26. How do you choose an embedding model, and how do you handle multilingual vs. language-specific embeddings?

• Choose based on task domain, required vector dimensionality (384 vs 1536), context length (512 vs 8192 tokens), and MTEB benchmark leaderboard score.
• **Multilingual Strategy**: Use native multilingual embedding models (e.g., text-embedding-3-large, bge-m3, cohere-embed-multilingual-v3.0) aligned across shared vector spaces when performing cross-lingual search.

4. Retrieval-Augmented Generation (RAG) & Vector Databases

Q27. How do you evaluate a RAG system using non-LLM metrics?

Separate the RAG pipeline into two independent evaluation layers:
1. **Retrieval Layer Evaluation (Information Retrieval Metrics)**:
• **Precision@K**: Fraction of top-K retrieved documents that are relevant.
• **Recall@K**: Percentage of all relevant ground-truth documents captured in top-K.
• **MRR (Mean Reciprocal Rank)**: Evaluates how high up the first relevant document is ranked (1 / rank).
• **NDCG (Normalized Discounted Cumulative Gain)**: Evaluates ranking quality with graded relevance scores.
2. **Generation Layer Evaluation (Text Matching Metrics)**:
• **Exact Match (EM)** & **F1-Score**: Token overlap between generated answer and ground-truth text.
• **ROUGE-L / BLEU**: N-gram overlap for summarization and text generation.

Q28. What are advanced RAG patterns (Dense, Sparse, Hybrid search, Parent Retrieval, and Reranking)?

• **Dense Search**: Vector similarity (Cosine/Inner Product) capturing semantic meaning.
• **Sparse Search**: Exact keyword matching using BM25 / TF-IDF.
• **Hybrid Search**: Combines Dense + Sparse scores using Reciprocal Rank Fusion (RRF).
• **Parent-Child Retrieval**: Decouples retrieval chunk size from context chunk size. Searches small child chunks (e.g., 128 tokens) for precision, but passes large parent sections (e.g., 1024 tokens) to the LLM.
• **Cross-Encoder Reranking**: Re-scores top-20 retrieved candidates using a joint cross-encoder model (e.g., Cohere Rerank) to produce top-3 high-confidence contexts.

Q29. How do you ingest and save 1,000 PDFs into a vector database?

1. **Parallel Text Extraction**: Process files in multi-threaded batches using high-speed parsers (PyMuPDF, pdfplumber). Use OCR engines (PaddleOCR/Tesseract) for scanned image pages.
2. **Cleaning & Metadata Tagging**: Strip headers/footers. Attach metadata (doc_id, filename, page_num, created_date, sha256_hash).
3. **Chunking**: Apply Recursive Character or Semantic Chunking (e.g., 512 tokens with 10% overlap).
4. **Batch Embedding & Upsert**: Embed text chunks in parallel GPU batches and bulk upsert vectors into vector DB (Qdrant/Pinecone/Milvus) with HNSW indexing.

Q30. How do you handle content updates when source PDFs are modified?

• **Document Hashing**: Store SHA-256 hash of each source file during initial ingestion.
• **Change Detection**: When a PDF is modified, compare its new hash against stored records.
• **Atomic Re-indexing**: Execute a database transaction: delete all existing vectors matching doc_id, re-chunk/re-embed the updated PDF, and upsert new vectors.
• **Cache Purging**: Clear semantic query cache entries linked to updated document IDs.

Q31. A previously well-performing RAG system suddenly starts hallucinating — how do you debug it?

1. **Inspect LLM Provider Endpoint**: Did the API update default model version, temperature, or system prompt?
2. **Isolate Retrieval vs. Generation**: Manually feed retrieved chunks directly into the LLM. If answer is wrong --> LLM/Prompt issue. If retrieved chunks lack factual content --> Retrieval/Embedding issue.
3. **Audit Recent Data Imports**: Check if recent PDF ingestion introduced corrupted, blank, or contradictory documents.
4. **Check Vector Store Index**: Verify vector DB connection, index build status (HNSW truncation), or dimension mismatch.

Q32. What if required PDF information is not included in any retrieved chunk?

• Increase top-K retrieval parameter (e.g., from top-3 to top-10 chunks) paired with a Reranker.
• Decrease chunk size or increase chunk overlap to avoid cutting sentences across boundaries.
• Implement Parent Document / Sentence Window Retrieval.
• Use Multi-Query Expansion (generate 3 rephrased query variations via LLM and combine retrieved results).

Q33. How do you extract text accurately from Arabic PDFs and scanned documents with embedded images?

• **RTL & BiDi Text Handling**: Arabic requires Right-to-Left (RTL) text reshaping (python-bidi, arabic_reshaper) to prevent reversed letter connections.
• **Scanned Images / OCR**: Use specialized Arabic OCR models (PaddleOCR, EasyOCR, or Cloud Vision APIs) configured for Arabic script.
• **Layout Analysis**: Use layout-aware tools (Unstructured, LayoutParser) to separate text blocks from image figures before passing to OCR.

Q34. What is a vector database, why is it better than a traditional database for semantic search, and how do you choose one?

• **Vector DB** stores high-dimensional vector embeddings and indexes them using Approximate Nearest Neighbor (ANN) search algorithms (HNSW, IVF-PQ) for sub-millisecond similarity search.
• Traditional SQL/NoSQL databases index exact scalar strings/numbers using B-Trees, making semantic similarity searches (O(N) dot products) prohibitively slow at scale.
• **Selection Criteria**: Latency requirements, hybrid filtering support, cloud managed vs self-hosted, payload indexing, and memory overhead.

Q35. What is the difference between FAISS and Qdrant? Can Qdrant be used locally?

• **FAISS (Facebook AI Similarity Search)**: In-memory C++/Python vector search *library*. Ultra-fast vector matrix operations, but lacks database features (no persistent storage, multi-tenancy, or payload filtering).
• **Qdrant**: Production-grade vector *database* written in Rust. Features persistent storage, rich JSON payload filtering, REST/gRPC APIs, and distributed clustering.
• **Local Execution**: Yes! Qdrant can run locally via Docker container, in-memory Python mode (QdrantClient(":memory:")), or local disk storage mode (QdrantClient(path="./qdrant_data")).

5. Agentic AI Systems & Architecture

Q36. What are the key design patterns in Agentic AI systems?

• **Single-Agent ReAct**: Interleaves Reasoning (Thought) and Action (Tool Call) loops.
• **Planner-Executor**: Explicit Planner agent decomposes high-level goals into sub-tasks; Executor agents run tasks sequentially.
• **Supervisor-Worker (Hierarchical)**: Supervisor agent delegates sub-problems to specialized worker agents and synthesizes output.
• **Multi-Agent Debate / Peer Review**: Multiple agents critique each other's outputs to improve accuracy.
• **Router Pattern**: Light classifier routes incoming user prompts to dedicated specialist pipelines.

Q37. What are the types of memory used in AI Agents?

• **Short-Term Memory**: In-context chat window buffer / sliding history passed to the LLM prompt.
• **Long-Term Memory**: Vector store or Key-Value database persisting user preferences, past interactions, and episodic facts across sessions.
• **Procedural Memory**: System instructions, agent tool schemas, and predefined workflow execution graphs (e.g., LangGraph state).

Q38. What is the Model Context Protocol (MCP) and why is it important?

• **MCP (Model Context Protocol)** is an open standard designed by Anthropic that standardizes how AI applications connect securely to external tools, databases, and local file systems.
• Prevents fragmented, custom API integrations by providing a uniform client-server protocol for tool discovery, prompt templates, and context streaming.

Q39. When would you choose an API-based approach over a Message Queue, and vice versa?

• **API (Synchronous Request-Response)**: Real-time user interactions requiring sub-second response (e.g., user authentication, interactive chat, instant search suggest).
• **Message Queue (Asynchronous Event-Driven)**: Long-running computational tasks (e.g., batch document embedding, video generation, model fine-tuning, PDF OCR) or workloads requiring rate-limiting, load leveling, and dead-letter queue (DLQ) retry policies.

Q40. How do you handle confidential data in AI chatbots without leaking it to third-party LLMs?

• **PII Anonymization & Redaction**: Run local Regex/NER scrubbers (e.g., Microsoft Presidio) to mask emails, SSNs, names, and credit cards before API transmission.
• **Local / On-Premise LLMs**: Deploy open-weight models (Llama 3, Mistral) on private VPC/infrastructure.
• **Enterprise Zero-Data-Retention (ZDR) Contracts**: Use enterprise endpoints (Azure OpenAI / AWS Bedrock) with strict legal non-training policies.

6. MLOps, Dataiku & System Design

Q41. What do you know about Dataiku (Dataiku DSS)?

Dataiku is an all-in-one platform for data science, analytics, and machine learning. It helps data scientists and business analysts work together in one workspace.
• **Visual & Code Interface**: Allows non-coders to use visual drag-and-drop tools while developers write Python, R, or SQL.
• **Data Preparation**: Connects directly to databases (Snowflake, SQL, S3) and cleans data efficiently.
• **AutoML & Deployment**: Builds models automatically and deploys them as REST API endpoints with one click.
• **MLOps & Governance**: Monitors model health, data drift, lineaging, and security access controls.
• **Generative AI (LLM Mesh)**: Safely integrates LLMs (OpenAI, Claude, local models) to build RAG applications with guardrails.

Q42. What strategies can you use to detect and respond to model drift after deployment?

• **Data Drift**: Shift in feature inputs P(X). Detect using Population Stability Index (PSI > 0.2), Kolmogorov-Smirnov (KS) test, or Wasserstein distance.
• **Concept Drift**: Shift in relationship P(Y|X). Track live accuracy/F1-score degradation against ground-truth feedback.
• **Response**: Trigger automated retraining on recent rolling windows, deploy retrained model in Shadow Mode, or fall back to rule-based heuristics.

Q43. How do you monitor an AI/ML application in production?

• **System Telemetry**: Latency (p95/p99), GPU/CPU memory utilization, throughput (RPS), error rates.
• **Data Quality & Drift**: Feature null rates, schema validation (Great Expectations), PSI feature drift.
• **LLM Operational Metrics**: Token usage, cost per request, hallucination detection (TruLens/Ragas), toxicity/guardrail block rate.

Q44. How would you design a recommendation system for an e-commerce platform?

• **Multi-Stage Pipeline**:
  1. *Candidate Generation (Retrieval)*: Collaborative filtering / Two-Tower Vector Embeddings narrow 10M items to top-500 candidates.
  2. *Scoring & Ranking*: Deep Neural Network / XGBoost scores candidates using user features, item features, and contextual data.
  3. *Re-ranking & Business Rules*: Apply deduplication, out-of-stock filtering, and diversity constraints.
• **Evaluation**: Offline (NDCG@K, Recall@K) --> Online A/B Test (CTR, Revenue per User).

Q45. What is a feature store, and how does it prevent training-serving skew?

• A central repository (e.g., Feast, Tecton) that standardizes feature definitions across training and inference.
• Dual storage engine: Low-latency Key-Value store (Redis/DynamoDB) for online real-time inference, and columnar data warehouse (Snowflake/BigQuery) for offline training. Eliminates training-serving skew.

Q46. How would you scale model inference to handle millions of requests per day?

• **Model Optimization**: Quantization (INT8/FP8 via AWQ/GPTQ), Knowledge Distillation, TensorRT compilation.
• **Serving Engines**: Use high-throughput servers (vLLM, TGI, Triton Inference Server) with Continuous Batching and PagedAttention.
• **Infrastructure**: Horizontal Pod Autoscaling (K8s), CDN/Redis caching of common query embeddings, and asynchronous message queues for non-realtime batches.

7. Practical Coding, Debugging & Case Studies

Q47. Implement K-Means Clustering from scratch.

Algorithm Logic: Initialize K centroids randomly --> Assign each data point to nearest centroid (Euclidean distance) --> Recompute centroids as mean of assigned points --> Repeat until convergence.

```
import numpy as np

def kmeans(X, k, max_iters=100, tol=1e-4):
# 1. Random centroid initialization
idx = np.random.choice(len(X), k, replace=False)
centroids = X[idx]

for _ in range(max_iters):
# 2. Compute Euclidean distances to all centroids
distances = np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)
labels = np.argmin(distances, axis=1)

# 3. Update centroids
new_centroids = np.array([X[labels == i].mean(axis=0) for i in range(k)])

# 4. Check for convergence
if np.all(np.abs(new_centroids - centroids) < tol):
break
centroids = new_centroids

return centroids, labels
```

Q48. Write code to compute a confusion matrix without using libraries.

```
def compute_confusion_matrix(y_true, y_pred):
# Assuming binary classification (0 and 1)
tp = sum(1 for gt, pred in zip(y_true, y_pred) if gt == 1 and pred == 1)
tn = sum(1 for gt, pred in zip(y_true, y_pred) if gt == 0 and pred == 0)
fp = sum(1 for gt, pred in zip(y_true, y_pred) if gt == 0 and pred == 1)
fn = sum(1 for gt, pred in zip(y_true, y_pred) if gt == 1 and pred == 0)

# Returns 2x2 matrix: [[TN, FP], [FN, TP]]
return [[tn, fp], [fn, tp]]
```

Q49. How do you split time-series data for training and validation without data leakage?

• Never use random shuffle split! Always preserve chronological ordering.
• Use **Time-Series Walk-Forward Validation** (expanding window or rolling window). Train on past data up to time T, validate on [T + 1, T + K]. Move time cutoff forward iteratively.

Q50. Given a dataset with missing values, walk through your preprocessing steps.

1. **Quantify Missingness**: Check missing percentage per feature.
2. **Identify Mechanism**: Missing Completely at Random (MCAR), Missing at Random (MAR), or Missing Not at Random (MNAR).
3. **Imputation Strategy**:
   • *Numerical*: Median (robust to outliers) or KNN Imputer.
   • *Categorical*: Mode or new category "Missing".
   • Add a binary feature_is_missing indicator column to capture missingness signal.

Q51. Debug this: A model performs well in training but poorly in production — what could be wrong?

1. **Data Leakage**: Target-related features accidentally included in training dataset.
2. **Training-Serving Skew**: Preprocessing transformations applied differently in production API vs. offline training pipeline.
3. **Data Drift**: Live production input distribution shifted significantly from training data.
4. **Overfitting**: Model hyperparameter tuning overfit the validation set.

Q52. How do you communicate technical ML results to non-technical stakeholders?

• Frame results in business impact and financial terms (e.g., cost savings, user retention lift, time saved) rather than technical metrics (like loss or ROC-AUC).
• Use intuitive visual charts over statistical data tables.
• Clearly explain model confidence limits, trade-offs, and operational risks in plain language.